

Fondamenti di programmazione

Di: Edoardo Lo Iacono

Contents

1	I programmi	3
2	Installazione dell'ambiente di sviluppo	3
3	Le variabili	3
3.1	Standard di notazione delle variabili	4
3.2	Principali tipi di variabili	4
4	Le condizioni	5
5	I cicli	6
5.1	Il ciclo FOR	6
5.2	Il ciclo precondizionale - WHILE	7
5.3	Il ciclo postcondizionale - DO...WHILE	9
6	I vettori	9
7	Le stringhe	11
7.1	Concatenazione	11
7.2	Calcolo della Lunghezza	12
7.3	Confronto	12
7.4	Copia	12
8	Le strutture	13
9	Le funzioni	14

1 I programmi

I programmi informatici sono oggi materia comune per tutti quanti, sono programmi le applicazioni che usiamo sul nostro cellulare come i siti web su cui navighiamo. Compresa l'importanza di essi dobbiamo capire bene cosa significhi scrivere un programma per poter proseguire ed andare ad analizzare le diverse parti che lo compongono.

Possiamo definire un programma come un insieme di istruzioni tradotte in un determinato linguaggio di programmazione. Queste istruzioni sequenziali vengono comunemente chiamate **algoritmo**.

Queste istruzioni prendono solitamente in considerazione dei dati in ingresso, chiamati **input** per restituirli sotto forma di dati in uscita detti **output**. Per comprendere meglio questo concetto andiamo a vedere un paio di esempi.

- Un primo esempio slegato dal mondo dell'informatica ma molto attinente è quello di una ricetta, in questo caso i nostri dati in input saranno gli ingredienti presi separatamente, il nostro algoritmo sarà la ricetta vera e propria, ovvero le istruzioni da seguire e infine il piatto portato a termine sarà il nostro output.
- Dato un semplice problema che ci richiede di calcolare la media aritmetica di 3 numeri in ingresso, i nostri dati in input saranno i 3 numeri in quanto vengono inseriti dall'utente, l'algoritmo sarà l'insieme di operazioni che portano a calcolare la media, ovvero la somma dei 3 valori in ingresso divisa per 3 ($\frac{v1+v2+v3}{3}$), l'output sarà la media vera e propria.

Visti questi esempi possiamo notare come un'algoritmo sia quindi un insieme finito di azioni che risolvono un determinato problema (cucinare, calcolare la media) andando a trasformare dei dati in input (ingredienti, valori) in dati in output (piatto, media).

2 Installazione dell'ambiente di sviluppo

Per poter andare ad eseguire il nostro programma rendendolo "digeribile" dal computer è necessario andare a compilarlo, ovvero tradurlo in un linguaggio che sia comprensibile dai nostri computer: il linguaggio macchina. Dato che sarebbe troppo difficile per i programmatori scrivere codice in linguaggio macchina si occupa di questa conversione l'**IDE**, ovvero l'ambiente di sviluppo integrato il quale non è altro che un software che raggruppa al suo interno tutti gli strumenti necessari per scrivere, modificare, compilare, e fare il debug del codice.

Ad oggi l'IDE più importante è *visual studio code*, di proprietà di Microsoft, scaricabile al seguente link: [vscode](#).

3 Le variabili

Il modo più semplice per pensare ad una variabile è figurarsela come un **contenitore** allocato (ovvero memorizzato) nella memoria dell'elaboratore. Le variabili possono essere di due tipi in base al linguaggio di programmazione:

- **Tipizzate** = Dire che una variabile è tipizzata significa specificare che dentro il nostro contenitore sarà possibile inserire *solo* elementi di un determinato tipo, parleremo tra poco dei diversi tipi di variabile. Il tipo di elemento scelto viene in questi casi specificato in fase di creazione di una variabile.

- **Non tipizzate** = Al contrario delle variabili tipizzate non è necessario stabilire da subito quale tipo di elementi sarà possibile mettere all'interno della nostra variabile.

Per comprendere meglio il concetto teorico di variabile riprendiamo l'esempio della media fatto prima; in questo caso le nostre variabili saranno i 3 valori che diamo in ingresso le quali avranno al loro interno un valore di tipo numerico.

A ogni variabile è associato un nome detto **identificatore** che identifica in maniera univoca la variabile nel programma e, come detto un tipo il quale indica la natura degli elementi interni a questi contenitori. Sono chiamate variabili in quanto al loro interno sono presenti valori che possono cambiare durante l'esecuzione del programma, analoghe alle variabili ma con valori che rimangono invece statici troviamo le *costanti*.

La sintassi per dichiarare una variabile, comune alla maggior parte dei linguaggi di programmazione tipizzati è la seguente:

```
1 Tipo nomeVariabile = eventualeValore;
```

Il valore interno della variabile può essere definito staticamente mettendolo dopo l'uguale oppure dinamicamente nel codice, in questo caso la dichiarazione sarà soltanto Tipo nomeVariabile;

3.1 Standard di notazione delle variabili

Uno dei problemi più comuni per chi è alle prime armi nella programmazione di qualsiasi tipo è quello dovuto a delle incongruenze nei nomi delle variabili; prima di chiarire meglio quello che intendo specifichiamo una cosa: i nomi delle variabili sono del tutto **arbitrari**, questo significa che è a discrezione del programmatore l'identificatore che viene dato alla variabile. Proprio per questo è una buona norma usare dei nomi **parlanti**. Si definisce parlante un nome che renda facile comprendere lo scopo ed il contenuto di una variabile anche in assenza del resto del codice. Tornando al discorso iniziale, uno dei principali errori di programmatori principianti e non è quello di usare caratteri che non sono consentiti o possono creare problemi nel nome di una variabile, è importante tenere a mente che non vanno mai usati *spazi* ed *accenti*.

Per ovviare all'assenza di spazi, gli sviluppatori hanno trovato altri modi di scrivere i nomi delle variabili (e di altre entità che vedremo in seguito) nel codice, tra i più comuni troviamo:

Stile	Descrizione	Esempio
Camel Case	Inizio minuscolo, parole successive attaccate con iniziale maiuscola.	<i>ciaoMondo</i>
Snake Case	Parole minuscole separate da trattino basso.	<i>ciao_mondo</i>
Kebab Case	Parole minuscole separate da trattino normale.	<i>ciao - mondo</i>
Pascal Case	Simile al Camel Case, ma con la prima lettera maiuscola.	<i>CiaoMondo</i>

3.2 Principali tipi di variabili

I principali tipi di variabili, comuni a pressoché tutti i linguaggi di programmazione sono:

- **int**: Rappresenta i numeri interi (Integer)
- **double**: Rappresenta i numeri reali con virgola mobile

- **char**: Rappresenta un singolo carattere, si scrive tra apici singoli (")
- **string**: Rappresenta una stringa ovvero un insieme (o per meglio dire un array, concetto che vedremo meglio in seguito) di caratteri, si scrive tra doppi apici ("")
- **bool**: Rappresenta un valore binario che può essere 'true' oppure 'false'

4 Le condizioni

Per dare una definizione balisare abbiamo detto che un algoritmo è un insieme di operazioni eseguite in sequenza l'una dopo l'altra, questa definizione non è però del tutto vera, o per meglio dire non lo è sempre. Con l'entrata in gioco delle condizioni abbiamo parti di codice che vengono o meno processati in base alla validità o meno di una condizione. E' quindi normale trovare parti di codice che in alcune esecuzioni possono venire completamente ignorate.

Prima di approfondire il discorso sui comandi condizionali credo che sia necessario fare un breve ripasso di **logica**, in quanto ci tornerà molto utile in seguito: Tra i principali operatori logici troviamo la **AND**, espressa solitamente con il simbolo $\wedge$ e la **OR**, che possiamo dividere in una disgiunzione inclusiva (in latino *vel*), rappresentata con il simbolo $\vee$ e la disgiunzione esclusiva (in latino *aut* la quale accetta che solamente uno dei due valori sia vero e non entrambi). Poniamo di seguito le tavole di verità di questi due operatori:

AND

A	B	$A \wedge B$
V	F	F
V	V	V
F	V	F
F	F	F

OR

A	B	$A \vee B$
V	F	V
V	V	V
F	V	V
F	F	F

Fatto questo brevissimo richiamo alla logica, torniamo a parlare delle operazioni condizionali, nella maggior parte dei linguaggi di programmazione, il blocco condizionale si sviluppa nel seguente modo

```

1 if(condizione)
2 {
3     ... //Codice eseguito SOLAMENTE se la condizione tra parentesi risulta vera
4 }
5 else
6 {
7     ... //Codice eseguito SOLAMENTE se la condizione tra parentesi risulta falsa
8 }
9 //Codice eseguito sempre

```

Possiamo avere però anche dei casi più particolari in cui la nostra condizione si ramifica in più condizioni, pensiamo per esempio di voler applicare uno sconto se il numero di biglietti comprati è superiore a 5 ed uno sconto ancora maggiore se è superiore a 10, non ci basterà qui una sola condizione. In questo caso possiamo usare una struttura del seguente tipo:

```

1 if(nPartecipanti >= 10)
2 {
3     //Applico lo sconto del 50\%
4 }

```

```
5 else if(nPartecipanti >= 5)
6 {
7     //La condizione risulta falsa rispetto alla prima if e quindi fa parte dell'else, ma
8     poniamo qui una nuova condizione
9 }
```

Nel caso in cui si venga a creare una condizione per cui sono necessarie molte 'else if', conviene usare il blocco **switch**, che subisce leggere variazioni in ogni linguaggio di programmazione ma permette, in base al valore di una variabile di eseguire operazioni diverse tra di loro.

5 I cicli

In informatica usiamo un ciclo quando dobbiamo ripetere una medesima porzione di programma per più volte, vediamo di seguito le principali tipologie di ciclo.

5.1 Il ciclo FOR

Il ciclo for è una porzione di controllo iterativa che, come detto **ripete per un numero definito di volte una stessa porzione di programma**.

Il ciclo si basa su 3 elementi fondamentali:

1. **Variabile di ciclo**
2. **Condizione di uscita**
3. **Espressione iterativa**

Vediamo meglio queste tre componenti prendendo in esame la struttura di base del ciclo for nel linguaggio **c e derivati**:

```
1 int n = 5;
2 for(int i = 0; i < n; i++){
3     //Operazioni
4     printf("Ciao_mondo");
5 }
```

quello che ci aspettiamo da questo semplice blocco di codice (scritto come detto in C, solo a scopo esemplificativo) è di ricevere a schermo la scritta "Ciao_Mondo" per 5 volte, andiamo a commentare meglio questo codice per andare a comprendere il suo funzionamento.

Andiamo in particolar modo a commentare la riga 2, in cui è presente la struttura vera e propria del nostro ciclo; Vediamo all'interno delle parentesi tonde tre "sezioni" di codice divise l'una dall'altra dal ; queste tre sezioni rappresentano i 3 elementi fondanti di ogni ciclo for.

`int i = 0;` non è altro che la dichiarazione della nostra variabile di ciclo, abbiamo parlato in precedenza delle variabili: cosa sono e quali sono le loro tipologie, non mi ci soffermo quindi ulteriormente arrivati a questo punto, è però importante notare una cosa: **la variabile di ciclo può essere utilizzata solamente all'interno del ciclo stesso**. Parleremo meglio di *variabili locali e globali* nella sezione sulle funzioni, limitiamoci in questo momento a capire che già a partire dall'uso delle condizioni, il nostro codice si sviluppa con una **struttura a matryoska**, in linea generale quindi, ogni volta che troviamo delle parentesi graffe {} abbiamo una condizione per cui, quello che è dichiarato al

loro interno può essere letto SOLAMENTE al loro interno, mentre quanto viene dichiarato al loro esterno può essere letto anche dentro, non fossilizziamoci ora su questo concetto di cui parleremo più approfonditamente in seguito, è però fondamentale capire che quella variabile `i` sarà utilizzata solo all'interno delle parentesi e quindi solamente nella parte di codice ripetuta.

`i < n`; rappresenta la nostra condizione di uscita, abbiamo detto che un ciclo `for` premette di ripetere una determinata sezione di codice per un numero noto di volte, è grazie a questa sezione che noi sappiamo **quante volte il codice verrà ripetuto**. In maniera più specifica questa condizione ci dice se uscire dal nostro ciclo o continuare ad entrarvi, è ovviamente essenziale una condizione di uscita in quanto in sua assenza avremmo un ciclo infinito. **Quando la condizione di uscita è vera dobbiamo continuare a ciclare, quando risulta invece falsa possiamo uscire dal nostro ciclo**, abbiamo quindi inizializzato la nostra variabile di ciclo con il valore 0 e definito che, finché il valore della nostra `i` è minore di `n` dobbiamo continuare a fare quanto scritto nel blocco sottostante delimitato dalle parentesi graffe, manca ancora però un elemento importantissimo, ora come ora infatti la nostra variabile rimane sempre ferma al suo valore facendo sì che non potrà mai arrivare ad essere uguale o maggiore di `n`.

`i++` è la nostra funzione iterativa, in parole povere **cambia valore alla nostra variabile di ciclo**, idealmente l'operazione qui espressa (che in questo caso incrementa semplicemente la nostra variabile `i` di uno) viene eseguita dopo tutte le altre operazioni specificate ad ogni giro.

Ovviamente i valori inseriti nel blocco precedente sono del tutto esemplificativi, seppure la struttura del `for` sia spesso simile, capiremo meglio in seguito, parlando di vettori, come mai è comune far partire un ciclo da 0 e non come ci potremmo aspettare da 1, in genere trovo utile ricordare il valore attribuito ad `i` durante la dichiarazione come il punto di partenza ed il valore che inseriamo invece in relazione alla `i` nella seconda sezione come il punto di arrivo. Per comprendere meglio questo concetto vediamo due esempi che potranno tornare utili anche in fase di esercizio:

```
1 //Stampare i numeri da 1 a 10
2 for(int i = 1; i < 10; i++){
3     printf("%d",i); //Per comprendere meglio la sintassi si veda il pdf sul linguaggio c
4     , in generale sta solo stampando a schermo il valore di i
5 }
6
7 //Stampare i numeri da 10 a 0
8 for(int i = 10; i > 0; i--){
9     printf("%d",i);
10 }
```

In altri linguaggi come **python** la sintassi risulta notevolmente semplificata, troviamo infatti il ciclo `for` espresso nel seguente modo:

```
1 //Stampare i numeri da 1 a 10
2 for n in range(1, 10): //Se non viene specificato, l'incremento di uno in positivo
3     print n
```

5.2 Il ciclo precondizionale - WHILE

La grande differenza tra i cicli precondizionali e postcondizionali (di cui parleremo nel paragrafo successivo) con il `for` è che, mentre usiamo quest'ultimo quando siamo certi del numero di volte in cui dovremo ripetere una data azione, ci avvaliamo dei due precedenti quando vogliamo **ripetere il**

nostro blocco di codice finché è valida una determinata condizione, perciò il numero di cicli potrebbe (potrebbe!) non essere noto a priori.

Partiamo di seguito a parlare del ciclo **precondizionale**, l'enclitico *pre-* ci fornisce un'importante elemento per la comprensione di questo ciclo in cui **la condizione di uscita viene verificata prima del codice ripetuto**. Prima di addentrarci in maniera più tecnica sul ciclo while prendiamo in considerazione un esempio che nulla ha a che vedere con il mondo dell'informatica: Pensiamo infatti di voler accedere ad un museo a pagamento, all'ingresso ci verrà chiesto il nostro biglietto, se questo è stato acquistato correttamente potremo entrare e fare tutte le nostre operazioni, in caso contrario ci verrà richiesto di uscire. Questo meccanismo, che non è altro che quello che si applica con una normale condizione (`if...else`) è quello che fa il nostro ciclo, per prima cosa verifica la condizione di uscita, se questa è vera esce direttamente altrimenti entra nel ciclo.

Vediamo di seguito la sintassi di base di questo ciclo con, a seguire un semplice esempio:

```
1 while(condizione)
2 {
3     //Operazioni ripetute
4 }
```

```
1 //Stampare i numeri da 0 a 2
2 int i = 0;
3 while(i < 3)
4 {
5     printf("%d",i);
6     i++;
7 }
```

Notiamo come quando usiamo il ciclo while **la variabile di ciclo non sia gestita direttamente all'interno dello stesso**, ma sia anzi compito nostro andare a dichiararla fuori (per far sì che si possa usare anche nella condizione) e incrementarla al suo interno. Molte volte, come in questo esempio i cicli pre/post condizionali ed il for sono intercambiabili, ci sono però molti casi in cui non è così, prendiamo un esempio (in `c#` per semplicità della sintassi):

```
1 string passwordCorretta = "segreto123";
2 string inserimento = "";
3
4 // Il ciclo continua all'infinito finché la condizione vera
5 while (inserimento != passwordCorretta)
6 {
7     Console.Write("Inserisci la password: ");
8     inserimento = Console.ReadLine();
9
10    if (inserimento != passwordCorretta)
11    {
12        Console.WriteLine("Password errata. Ritenta!");
13    }
14 }
15
16 Console.WriteLine("Accesso confermato!");
```


5.3 Il ciclo postcondizionale - DO...WHILE

Il ciclo postcondizionale va di paripasso con quello precondizionale visto precedentemente, sono infatti intercambiabili in molti casi. La caratteristica del ciclo postcondizionale è che **il blocco di codice da ripetere viene eseguito almeno una volta e successivamente viene fatto il nostro controllo**. Per comprenderlo in termini meno tecnici pensiamo all'insieme di azioni che dobbiamo fare per decidere se un piatto è di nostro gradimento o meno: prima lo assaggiamo e successivamente lo valutiamo, la stessa logica è alla base di questo ciclo. La sintassi basilare di questo ciclo è la seguente:

```
1  do
2  {
3  //Operazioni
4  }while(condizione)
```

Il ciclo precondizionale è prevalentemente usato nei casi in cui dobbiamo continuare a chiedere all'utente un dato che deve rispettare determinate condizioni, vediamo per esempio come chiedere in ingresso un numero positivo

```
1  int n;
2  do
3  {
4      printf("Inserisci n: ");
5      scanf("%d",&n);
6  }while(n < 0)
7  printf("%d",n);
```

questo esempio scritto in c ci permette di chiedere all'utente un numero, se il numero inserito è positivo viene stampato a schermo, in caso contrario si continuerà a chiedere fino a quando questa condizione verrà rispettata.

6 I vettori

Abbiamo parlato al capitolo 3. delle variabili considerando sempre e solo in maniera singola, ovvero capaci di contenere *un solo* elemento alla volta. Ci capita spesso tuttavia di dover gestire un **gruppo** di dati della stessa entità, entrano in questi casi in gioco i vettori, chiamati anche con il nome **array**. Ampliamo la metafora delle variabili = Scatole per comprendere meglio la natura dei vettori, possiamo infatti più propriamente considerare le variabili come i singoli armadietti ed i vettori come l'insieme di armadietti di una stessa classe o squadra sportiva.

Un vettore è una **collezione di variabili dello stesso tipo**, memorizzate in posizioni di memoria contigue. Possiamo immaginarlo come una riga di armadietti numerati, dove ogni armadietto contiene un dato. E' necessario sottolineare che questi armadietti sono **numerati** in quanto sarebbe altrimenti impossibile andare ad accedere al dato singolo di cui abbiamo bisogno. Per questo motivo i vettori si avvalgono di **indici**. E' fondamentale ricordare che gli indici di un vettore vanno da 0 alla lunghezza del vettore stesso - 1, quindi $i \in [0, n - 1]$. Dobbiamo considerare i vettori e le operazioni ad esse associate come non come operazioni attribuite ad una grande variabile ma piuttosto come un insieme di operazioni ripetute su più variabili, per questo i vettori vanno spesso di paripasso con i cicli.

Abbiamo quindi definito il vettore e chiarito come sia necessario fare riferimento ad un indice per

poter accedere ad un singolo elemento contenuto in esso, approfondiamo di seguito la sintassi generale dei vettori in modo da comprendere meglio questa dinamica con anche alcuni esempi:

Inizializzazione di un vettore Nella maggior parte dei linguaggi di programmazione i vettori si inizializzano in maniera molto simile a quella che abbiamo già affrontato parlando delle variabili, anche i vettori sono infatti tipizzati o meno, presentano un nome (completamente arbitrario) che ci permette di identificarli e possono avere già in fase di inizializzazione eventuali valori, quello che viene però aggiunto è:

- Il fatto che sia un vettore
- La sua lunghezza, ovvero quanti spazi contigui in memoria andare ad allocare

prendiamo quindi degli esempi in c per vedere la sintassi, abbiamo detto che in generale l'inizializzazione si compone nel seguente modo:

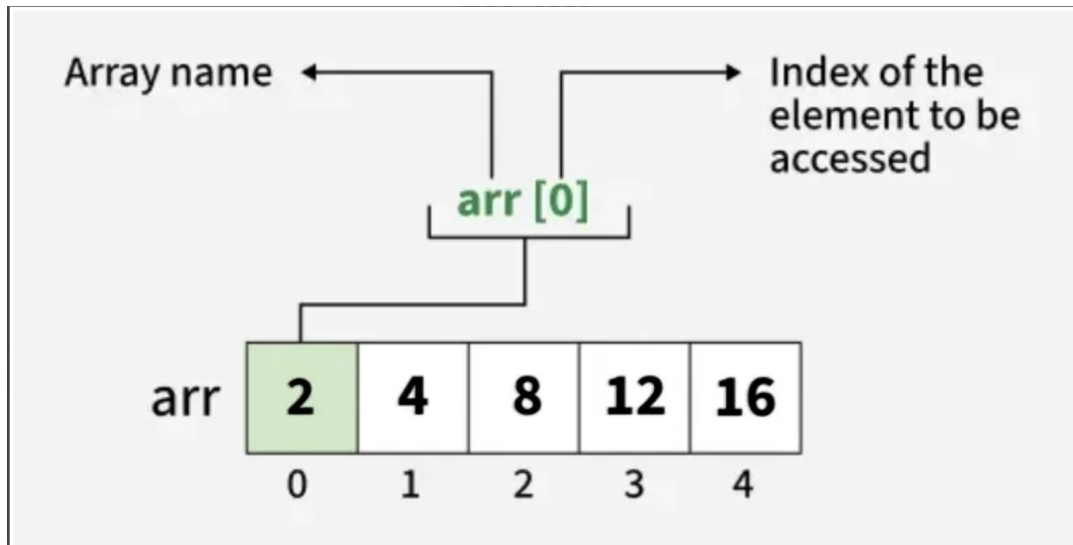
```
1 Tipo nomeVettore[Lunghezza del vettore] = {v1,v2,v3,...};
```

Per una maggiore comprensione ed un più vasto corpus di esempi si faccia riferimento alle seguenti fonti: [Fonte 1](#), [Fonte 2](#). Oltre a leggere la documentazione del linguaggio di programmazione di proprio interesse.

```
1 //Inizializzazione statica
2 int v1[] = {1,2,3,4,5};
3 int v2[3] = {0};
4 //Inizializzazione dinamica
5 int v3[n];
6 for(int i = 0; i < n; i++)
7 {
8     v3[i] = valore
9 }
```

Notiamo come, nel primo caso mettendo dei valori statici non sia necessario inserire la lunghezza del vettore in quanto viene calcolata automaticamente in base ai valori inseriti. Nel secondo caso come, aggiungendo la lunghezza ed un solo valore tutti gli altri verranno popolati allo stesso modo, v2 sarà infatti 0,0,0. con v3 vediamo invece il caso più comune, ovvero come caricare un vettore dinamicamente, al posto di "valore" ci vorrà ovviamente una richiesta all'utente piuttosto che un numero casuale.

E' necessario tenere bene a mente la distinzione tra indice e elemento contenuto nel vettore, come mostrato bene nella seguente immagine:



7 Le stringhe

Come abbiamo visto al paragrafo 3.2. Le stringhe sono uno dei principali tipi di variabili, possiamo ora comprendere meglio come le stringhe non siano altro che un **vettore di caratteri** gestito dai vari linguaggi di programmazione in maniera autonoma, se ci pensiamo in linea generale infatti una parola non è altro che l'insieme di diverse lettere unite tra loro seguendo delle regole sintattiche e morfologiche. Possiamo infatti inizializzare una stringa in modi diversi ma analoghi come:

```
1 char saluto[] = {'c','i','a','o','\0'};  
2 string saluto = "Ciao";
```

Si noti come nel caso in cui la stringa si voglia dichiarare esplicitamente come array di caratteri sia necessario aggiungere al termine il carattere `\0`, il quale permette al computer di capire che la stringa è finita.

In molti linguaggi di programmazione **è necessario includere la libreria string** per poter usare le stringhe e compiere delle operazioni più articolate, questa libreria ci permette infatti non solo di dichiarare in maniera implicita il vettore di caratteri come abbiamo visto nell'esempio sopra, ma anche di fare alcune operazioni con queste stringhe.

E' necessario come sempre fare riferimento alla documentazione del linguaggio di programmazione di proprio interesse, vediamo però di seguito alcune delle principali operazioni che possiamo fare con le stringhe in c e, di conseguenza in maniera simile nei linguaggi da esso evoluti:

7.1 Concatenazione

Il concatenamento è l'operazione che permette di **unire due stringhe** una di seguito all'altra per formarne una nuova. In C si utilizza la funzione `strcat`, mentre in linguaggi più moderni come il C++ si può usare semplicemente l'operatore matematico `+`.

```
1 // Esempio in C++ (Linguaggio evoluto)  
2 string a = "Ciao ";
```

```
3 string b = "Mondo";
4 string ab = a + b; // Risultato: "Edoardo Mondo"
5
6 // Esempio in C (Linguaggio base)
7 char s1[20] = "Ciao ";
8 char s2[] = "Mondo";
9 strcat(s1, s2); // s1 diventa "Ciao Mondo"
```

7.2 Calcolo della Lunghezza

Questa funzione serve a sapere **quanti caratteri** compongono la stringa, escludendo il carattere terminatore `\0`.

```
1 string parola = "Elettronica";
2 int lunghezza = parola.length(); // In C++ restituira' 11
3
4 // In C si usa strlen()
5 char parolaC[] = "Elettronica";
6 int lunghezzaC = strlen(parolaC);
```

7.3 Confronto

A differenza dei numeri, non sempre possiamo confrontare due stringhe con l'operatore `==` (specialmente in C). La libreria ci permette di capire se due stringhe sono identiche o quale viene prima in ordine alfabetico.

```
1 string p1 = "mela";
2 string p2 = "mela";
3
4 if(p1 == p2) {
5     // In C++ questo confronto e' permesso dalla libreria string
6     printf("Le parole sono uguali");
7 }
8
9 // In C si usa strcmp(s1, s2)
10 // Restituisce 0 se sono identiche
11 if(strcmp(stringa1, stringa2) == 0) {
12     // Stringhe uguali
13 }
```

7.4 Copia

Dato che una stringa è un array, non possiamo assegnare un valore a un altro con un semplice `s1 = s2` nei linguaggi di basso livello. Dobbiamo "trasferire" il contenuto da un contenitore all'altro.

```
1 // In C++ e' immediato
2 string originale = "Testo";
3 string copia = originale;
4
5 // In C dobbiamo usare strcpy
```

```

6 char sorgente[] = "Testo";
7 char destinazione[10];
8 strcpy(destinazione, sorgente);

```

Operazione	Funzione in C (<string.h>)	Stile C++ (<string>)
Concatenazione	strcat(s1, s2)	s1 + s2
Lunghezza	strlen(s1)	s1.length()
Confronto	strcmp(s1, s2) == 0	s1 == s2
Copia	strcpy(s1, s2)	s1 = s2

8 Le strutture

Le strutture sono un modo per raggruppare logicamente un insieme di informazioni collegate, trattandole come un'entità unica, potremmo in realtà fare a meno di esse ma ci permettono di rendere il nostro codice molto più ordinato e comprensibile, per capire bene questo concetto facciamo un esempio: consideriamo di voler gestire in un nostro progetto un insieme di studenti, ogni studente presenta delle caratteristiche comuni: un nome, un cognome, una classe di appartenenza ed una media aritmetica, possiamo tranquillamente considerare queste informazioni come delle variabili tipizzate a seconda dei principali tipi già affrontate ed inserirle all'interno di vettori paralleli in questo modo:

```

1 string nome = ["Mario", "Dario", "Luca", "Luigi", "Marco"];
2 string cognome = ["Rossi", "Verdi", "Bianchi", "Neri", "Marconi"];
3 int classe = [5,5,3,2,4];
4 double media = [7,8,6,5,6];
5
6 //Stampo i dati del primo utente
7 Console.WriteLine("Nome: " + nome[0])
8 Console.WriteLine("Cognome: " + cognome[0])
9 Console.WriteLine("Classe: " + classe[0])
10 Console.WriteLine("Media: " + media[0])

```

questa logica funziona ma è superflua e può creare diversi problemi nell'interazione con i dati, dato che ogni studente presenta le stesse caratteristiche possiamo creare una struct studente. Questa struct prenderà il ruolo di un nuovo tipo di dato che potremo usare a piacere e che conterrà al suo interno tutte quelle informazioni prima richieste. per accedere all'informazione in lettura o in scrittura sarà sufficiente indicarla con il . (variabile.attributo). Prima di vedere l'esempio fatto con le struct vediamo come definire una struct:

```

1 struct nomeStruct{
2     tipo membro1;
3     tipo membro2;
4     ...
5     tipo membrox;
6 }

```

nel nostro esempio potremmo voler creare una struct di questo tipo:

```

1 struct studente{
2     string nome;

```

```
3     string cognome;
4     int classe;
5     double media;
6 }
```

Inserendo staticamente un valore nella creazione di una struct, tutti gli elementi facenti parte di essa acquisiranno a priori quel valore (es. se mettessi `double media = 6.0`; tutti gli studenti che andrò a creare avranno a priori la media del 6). Come abbiamo detto, la struct, una volta inizializzata è considerabile al pari di un tipo come `int` o `char`, per questo dovremo andare ad inizializzare gli elementi (variabili o vettori) di quel determinato tipo:

```
1     studente s;
2     s.nome = "Maria";
3     s.cognome = "Verdi";
4     s.classe = 5;
5     s.media = 8.6;
6
7     Console.WriteLine(s.nome) //Maria
8     ...
```

possiamo ovviamente applicare lo stesso ragionamento anche con i vettori, essendo questi un insieme di variabili:

```
1     studente studenti[] = new studente[3]
2     for(int i = 0; i < 3; i++)
3     {
4         studente s;
5         s.nome = ...;
6         s.cognome = ...;
7         s.classe = ...;
8         s.media = ...;
9
10        studenti[i] = s;
11    }
12 }
```

Comprendere bene il concetto alla base delle strutture ci permette di rendere il nostro codice più pulito ed ordinato, facendoci mettere un primo passo verso un concetto che vederemo in seguito, ovvero quello della notazione json.

9 Le funzioni

Le funzioni in informatica rappresentano un insieme di codice con un nome univoco che svolge dei compiti precisi, possono avere dei valori in input e dei valori in output. La definizione fatta ora di funzione richiama fortemente quella vista all'inizio di questo manuale sul concetto stesso di algoritmo infatti, nonostante ne parliamo solamente ora noi stiamo lavorando con le funzioni già dal primo codice scritto, prendiamo per esempio il codice basilare (e solitamente di partenza) di un linguaggio come `c++`:

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello World!";
5     return 0;
6 }
```

`int main()` rappresenta infatti il nostro **entry point**, l'entry point è una funzione presente in tutti i linguaggi di programmazione (chiamata solitamente `main`) che viene eseguita quando viene avviato il programma, come detto abbiamo fino ad ora lavorato dentro ad una funzione senza necessariamente sapere cosa fosse.

Oltre all'entry point è possibile da parte del programmatore creare delle funzioni per degli scopi precisi, capito quindi che la funzione non è altro che un blocco di codice, perché può essere utile crearla?

- **Chiarezza** → In primo luogo può essere utile creare diverse funzioni nei nostri codice per una questione di organizzazione ed ordine, tecnicamente (AD UN LIVELLO BASILARE) nulla ci vieta di scrivere tutto quanto dentro al nostro `main` ma sarebbe confusionario andare a ritrovare una determinata parte di codice in caso di revisione o nel caso in cui fosse necessario eseguire un debug.
- **Riutilizzabilità** → Capita spesso in un codice di dover compiere una stessa operazione più volte in contesti diversi; in questo caso è preferibile inserire questa sezione di codice all'interno di una funzione che potrà essere richiamata ogni volta che sarà necessario, evitando di riscrivere le stesse istruzioni. Questo aspetto potrebbe sembrare simile all'utilizzo di un ciclo, poiché anche un ciclo permette di evitare ripetizioni, vi è però una grande differenza: il ciclo serve a eseguire automaticamente più volte uno stesso blocco di codice secondo una certa condizione, mentre una funzione serve a definire un'operazione specifica che può essere richiamata in punti diversi del programma, anche in momenti e contesti differenti.

Riconosciamo una variabile dalla presenza delle parentesi tonde `()` dopo precedute dal nome della variabile, notiamo inoltre che di fianco al nome `main` è presente un "elemento" che già conosciamo, viene infatti specificato il tipo della funzione.

Esistono 2 categorie di funzioni:

- **Le funzioni tipizzate** → Le funzioni in cui viene specificato il tipo vengono dette tipizzate, queste funzioni presentano al loro interno anche un comando `return` che andrà a ritornare dalla funzione un dato del tipo specificato, possiamo dire quindi che queste funzioni restituiscono necessariamente qualcosa in output.
- **Le funzioni void** → Le funzioni void non necessitano di un `return`, possono andare a modificare i parametri in input o a fare altre operazioni ma non ritornano niente in output (NB. per capire bene questo concetto può essere utile associarlo alle funzioni matematiche, in questo senso, quando viene detto che non restituisce nulla in output non si intende necessariamente che non genera un output (es. stampare a schermo qualcosa) ma piuttosto che quella funzione non ritorna alcun dato all'entry point)

Indipendentemente dal fatto che una funzione sia tipizzata o void, per andarla dovremo prima di tutti richiamarla dal `main`, facendo attenzione in linguaggi di programmazione come `c` o `c++` a far

sì che sia messa al di sopra di esso (questa regola non vale però per tutti i linguaggi, in caso di dubbi fare riferimento alle apposite documentazioni), in modo da essere leggibile, vediamo un esempio:

```
1 #include <iostream>
2 void saluto(){
3     std::cout << "Ciao"
4 }
5 int main(){
6     std::cout << "Prova";
7     saluto();
8     saluto();
9     return 0;
10 }
11
12 //Prova
13 //Ciao
14 //Ciao
```

Abbiamo detto prima che le funzioni sono riconoscibili grazie alle parentesi tonde, queste rendono possibile il passaggio di parametri, i parametri non sono altro che variabili che dall'esterno passiamo all'interno della nostra funzione per essere viste e manipolate da essa. Prima di approfondire questo concetto facciamo un richiamo ad una cosa già detta in precedenza parlando delle variabili, le variabili agiscono e sono infatti visibili solamente nello spazio in cui vengono create (lo abbiamo visto per esempio nel for, la variabile di ciclo non è manipolabile al di fuori di esso), **se quindi andiamo a creare delle variabili interne ad una funzione esse saranno visibili solo all'interno di essa**, discorso in realtà che si farà più complesso e completo introducendo il concetto di classe. Questa stessa regola vale anche per il main, se creo una variabile nel main non sarà subito visibile all'interno delle nostre funzioni, per poter rendere questo possibile ci sono diverse soluzioni:

- **Passare le variabili interessate come parametri:** E' la soluzione più sicura in quanto non va a modificare la visibilità di una variabile sul piano globale, ci basterà andare a passare le variabili nel richiamo della funzione e andarli a inizializzare durante la dichiarazione della funzione, come nel seguente caso:

```
1 #include <iostream>
2 #include <string>
3 void saluto(std::string nome){
4     std::cout << "Ciao, " << nome
5 }
6 int main(){
7     saluto("Pino"); //Ciao Pino
8     saluto("Claudio"); //Ciao Claudio
9     saluto(); //Ciao
10    return 0;
11 }
```

è anche possibile, in fase di dichiarazione della funzione, andare a dichiarare dei parametri standard i quali verranno inseriti nel caso in cui non venisse inserito alcun parametro in fase di richiamo:

```
1 #include <iostream>
2 #include <string>
```



```
3 void saluto(std::string nome = "Marco"){
4     std::cout << "Ciao, " << nome
5 }
6 int main(){
7     saluto("Pino"); //Ciao Pino
8     saluto("Claudio"); //Ciao Claudio
9     saluto(); //Ciao Marco
10    return 0;
11 }
```

nel caso in cui si passi una variabile e si voglia cambiare il valore di essa in base a delle operazioni eseguite nella funzione è necessario passare il parametro come **riferimento**, è possibile mediante caratteri come `&` in c++ o `ref` in altri linguaggi di programmazione:

```
1 //Senza riferimento
2 #include <iostream>
3
4 void nPerDieci(int n){
5     n = n*10;
6 };
7
8 int main() {
9     int numero = 20;
10    std::cout << numero ; //20
11    nPerDieci(numero);
12    std::cout << numero; //20
13    return 0;
14 }
15
16
17 // CON riferimento
18 #include <iostream>
19
20 void nPerDieci(int &n){
21     n = n*10;
22 };
23
24 int main() {
25     int numero = 20;
26     std::cout << numero ; //20
27     nPerDieci(numero);
28     std::cout << numero; //200
29     return 0;
30 }
```

- **dichiarare le variabili globalmente** in modo che siano visibili e manipolabili da tutto il codice, questa soluzione risolve però ovviamente meno sicura ed affidabile. In linea generale, per dichiarare una variabile globale essa va dichiarata al di fuori di tutte le funzioni presenti